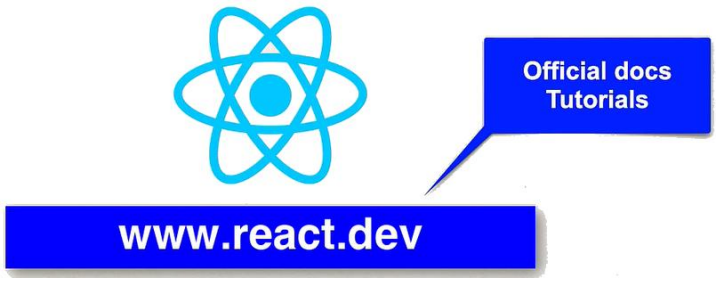
Getting Started with React and our Full-Stack Project

In this journey towards building a full-stack application using React and Spring Boot Java, we'll first delve into some essentials. This introduction is vital for those new to React or Single Page Applications (SPAs).

Introduction to React

React is a leading framework for building modern single-page applications. You can find comprehensive information, including tutorials and official documentation, at www.react.org. Bookmarking this site would be beneficial as it's filled with valuable insights.



Key Features of React

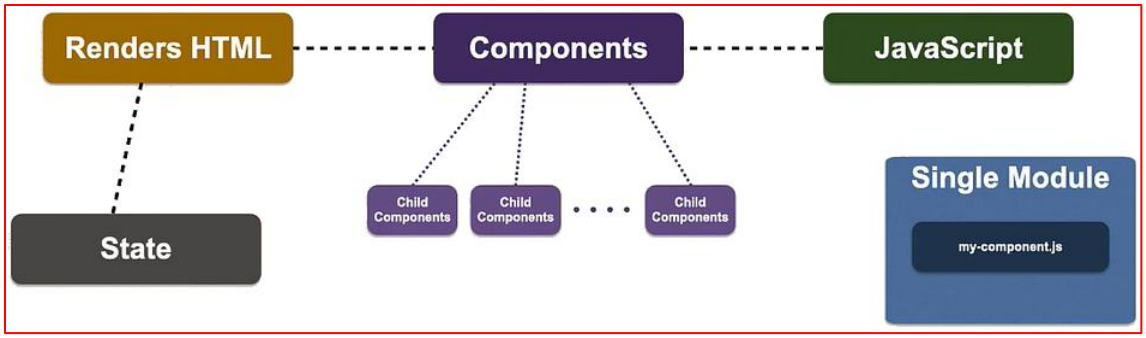
Here's an overview of React's standout features:

- **Component-Based Framework**: React's structure is built on reusable components.
- **Declarative Views**: Simplifies coding by allowing you to describe your UI's appearance.
- **Component Logic with JavaScript**: Write your logic in JavaScript, offering great flexibility.
- **Responsive Web Design Support**: React supports modern frameworks, including Bootstrap, Google Material Design, and others.

React Architecture: Components

React's architecture is essentially component-based. Components can contain child components, and those may contain even more child components. Here's how it works:

- **Components render HTML**: Users interact with various components on the screen.
- **Components handle state**: Each component has its state, which creates its lifecycle. Changes in the state cause the HTML to rerender.
- **Components are JavaScript files**: In your React application directory, each component is a JavaScript file, like `mycomponent.js` as illustrated in the following figure.



Key Terms in React Development

To grow within our React development skills, understanding these terms is essential:

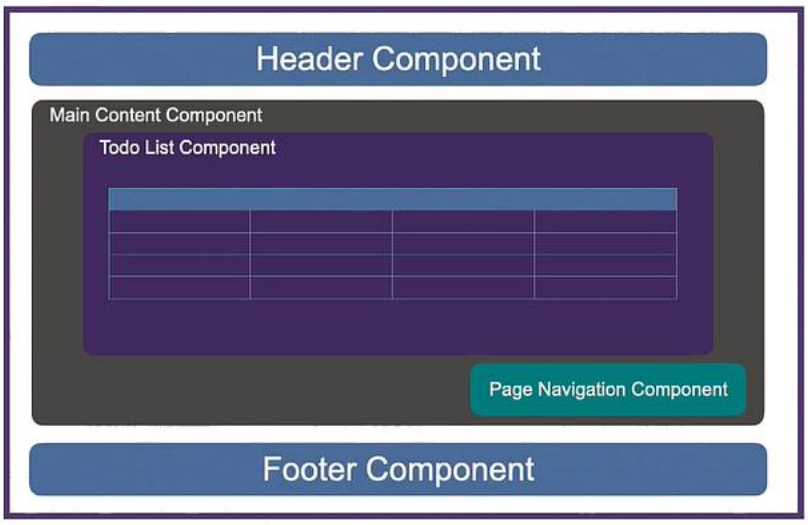
1. **Component**: The building block of a React application, consisting of the view (HTML) and application logic (JavaScript).
2. **Props**: A mechanism to pass data from parent to child components within the hierarchy.
3. **State**: Contains data about the component, and changes in state cause the component to rerender.
4. **Hooks**: A way to use local state with other React features without writing complex classes.
5. **Module**: A collection of related components that create a specific view, like a sales page.

Term	Definition
Component	Main player in an React application. Has two parts: 1. View for user interface 2. JavaScript that contains application logic / event handling for the view
Props	How we will pass data from a parent component to a child component
State	Contains data about the component. When the state changes, the component will re-render.
Hooks	A way to use local state and other React features without writing a new class
Module	A collection of related components, that create a specific view.

Visualizing a React Application UI

Imagine a purple rectangle frame as your client's PC monitor:

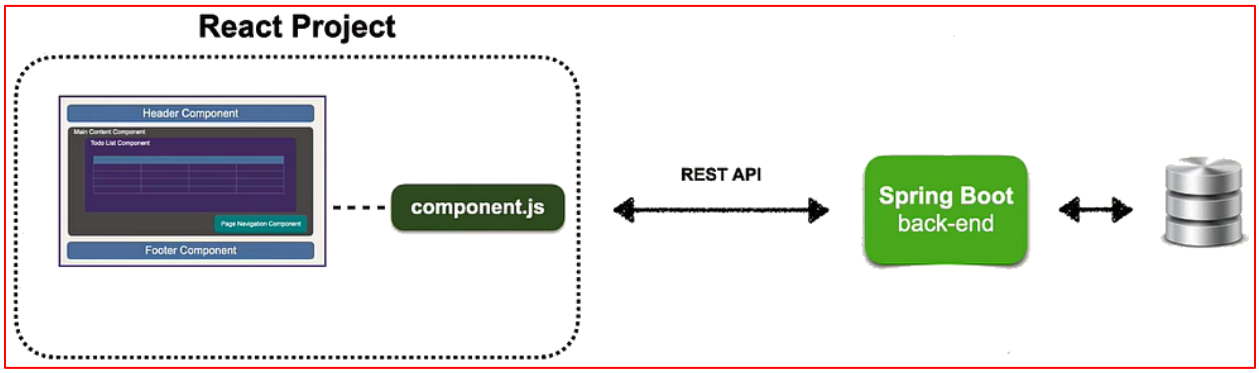
- **Header and Footer Components**: Common across all pages, like a navbar and footer.
- **Main Content Component**: The central part users interact with, and it may change from page to page.
- **Nested Components**: Inside the main content, you might have a to-do list and other sub-components.



Application Interaction

Here's a snapshot of how our React project will function:

- **Components structure**: Header, footer, main content, etc., forming the React project.
- **Communication**: Interaction over REST API with a Spring Boot backend, linked to a database.



React Project Composition

A React project comprises multiple files:

- **Components**: Containing HTML, JavaScript, CSS, and configuration files.
- **Media**: Including images and other media.
- **Miscellaneous**: Other necessary files contributing to building a production-ready application.

